HW 5 due this Friday, Mar 13

Quiz 7 coming soon

Class 14: Turing Machines

University of Virginia
CS3120: DMT2
https://weikailin.github.io/cs3120-toc
Wei-Kai Lin

Plan

Church-Turing Hypothesis

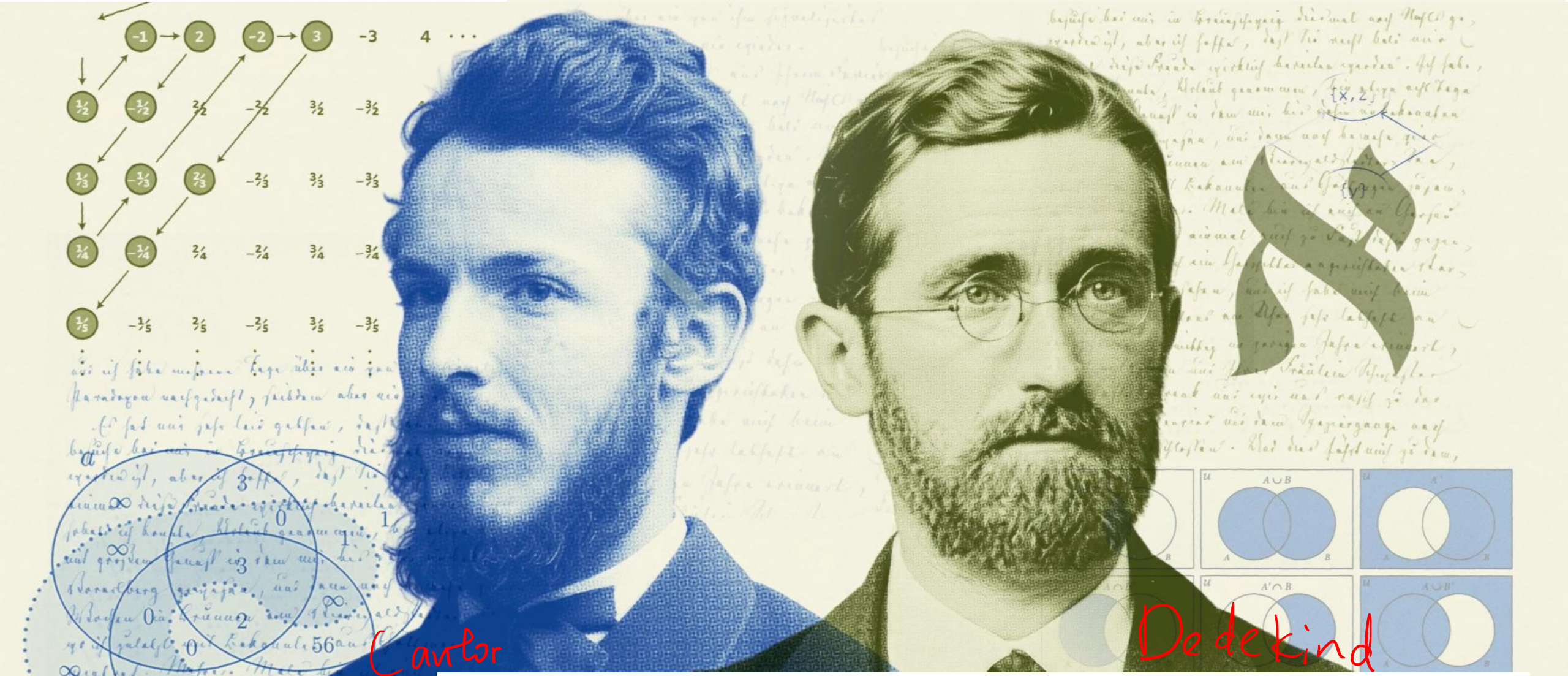
Turing Machines

Textbook [TCS] Section 5 and 7

https://introtcs.org/public/lec_06_loops.html
Loops and infinity

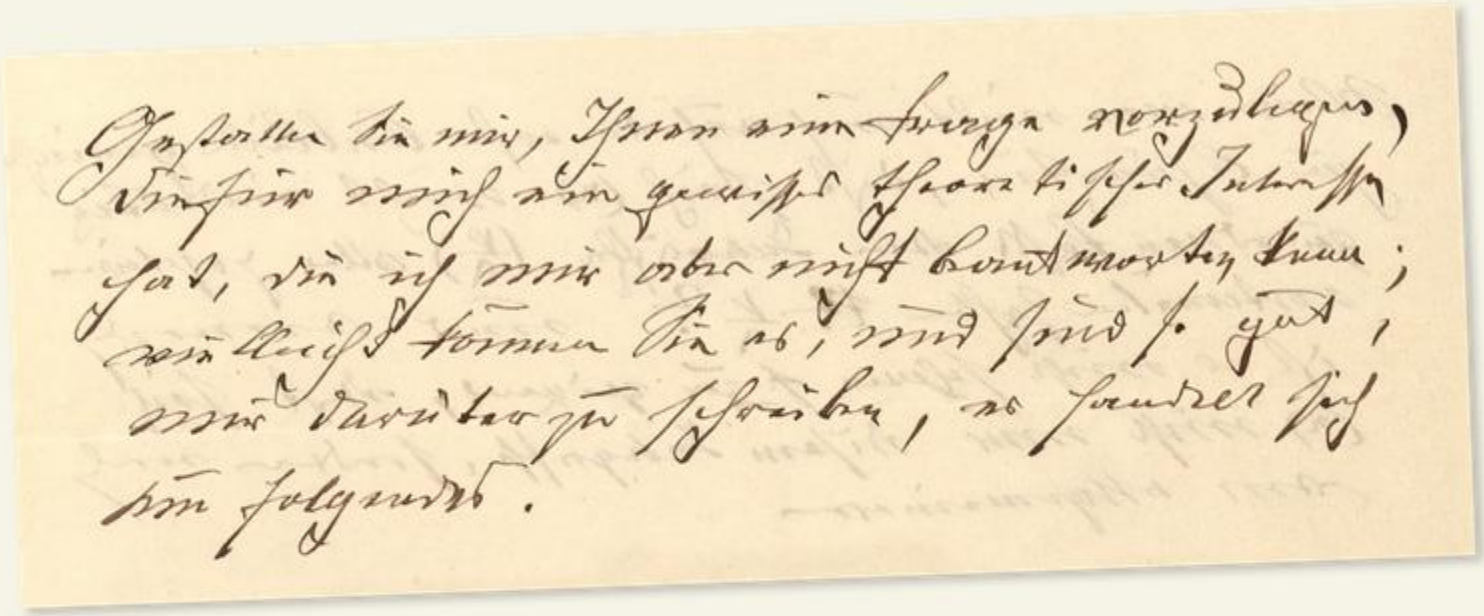
Quantamagazine

The Man Who Stole Infinity



https://www.quantamagazine.org/the-man-who-stole-infinity-20260225/ Feb 25, 2026

CANTOR TO DEDEKIND

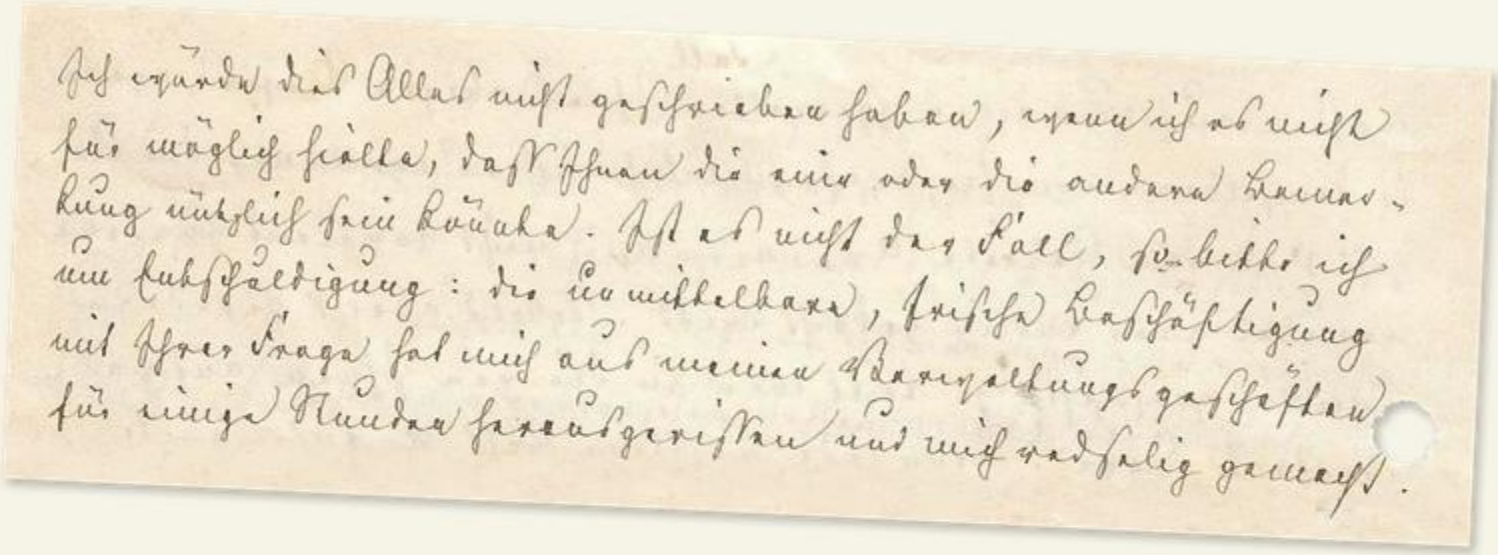


29 November 1873

"Allow me to put a question to you. It has a certain theoretical interest for me, but I cannot answer it myself; perhaps you can, and would be so good as to write me about it. It is as follows."

Question:
Were there more real numbers than whole numbers?

DEDEKIND TO CANTOR



Ich würde das Alles nicht geschrieben haben, wenn ich es nicht für möglich hielte, daß Ihnen die eine oder die andere Bemerkung nützlich sein könnte. Ist es nicht der Fall, so bitte ich um Entschuldigung: die unmittelbare, frische Beschäftigung mit Ihrer Frage hat mich aus meinen Berufsgeschäften für einige Stunden herausgerissen und mich redselig gemacht.

30 November 1873

"I would not have written all of this if I didn't think that one or another of these remarks might be useful to you. If that is not the case, I apologize: the immediate, fresh engagement with your question took me away from my duties for a few hours and has made me talkative."

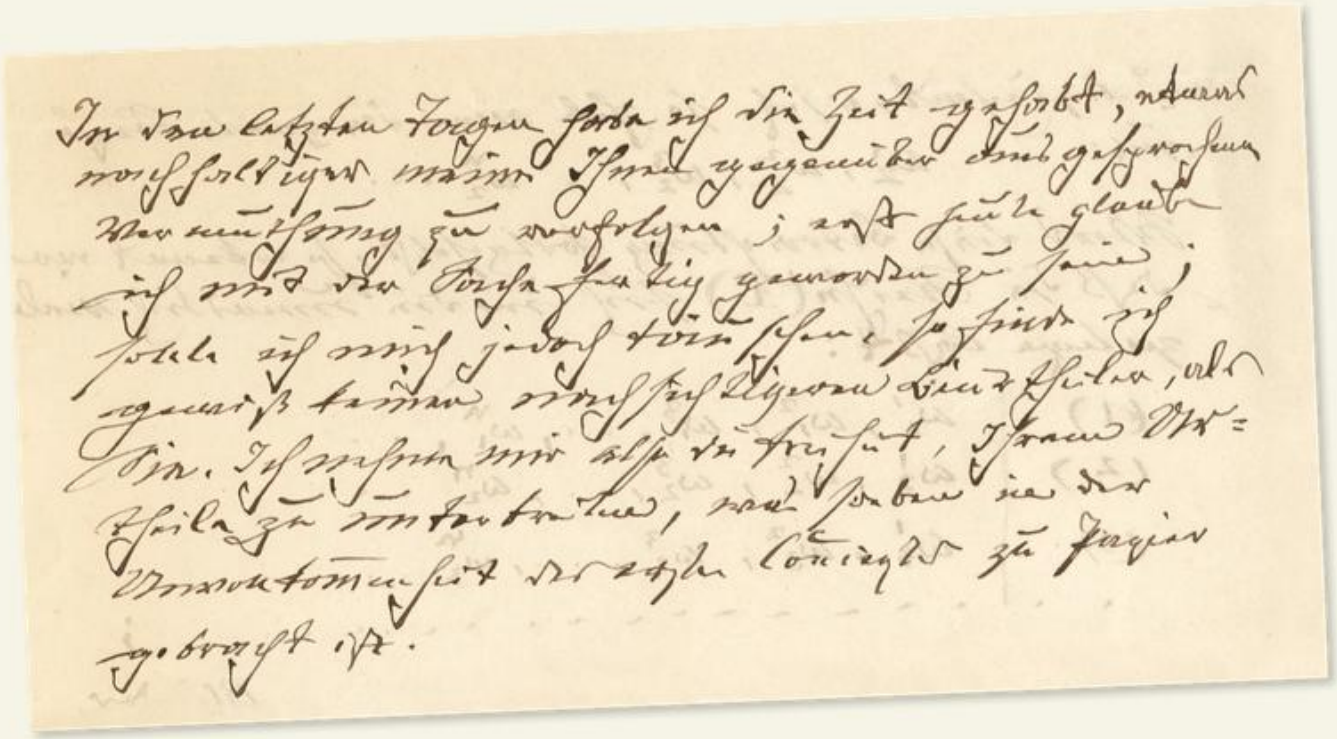
$0, 1, \frac{2}{3}, \sqrt{2}, \ldots$

Proof:

|algebraic numbers|
= |whole numbers |.

Algebraic number:
Solutions to whole number equations.

CANTOR TO DEDEKIND



7 December 1873

"In the last days I have had the time to pursue more thoroughly the conjecture I spoke to you about; only today do I believe myself to have finished with the thing; but if I should be deceiving myself, I should certainly find no more indulgent judge than you."

Proof:
|real numbers|
= |whole numbers |.
But it was unwieldy, convoluted.

Later

- Dedekind replied a simplification of Cantor's Dec-7 proof
- Cantor simplified Cantor's Dec-7 proof, at same time

- Dec 24, 1873: Cantor submitted to *Crelle*, consisting
  - Proof of |algebraic numbers| = |whole numbers |.
  - Proof of |real numbers| > |whole numbers |.
  - Both are now found in the letters from Dedekind. The only physical letters has been kept by Cantor's family.

Georg Cantor: Ueber eine Eigenschaft des Inbegriffes aller reellen algebraischen Zahlen.
(On a property of the class of all real algebraic numbers)
Journal für die Reine und Angewandte Mathematik 77 (1874), pp 258-62

Ueber eine Eigenschaft des Inbegriffes aller reellen algebraischen Zahlen.

(Von Herrn *Cantor* in Halle a. S.)

Unter einer reellen algebraischen Zahl wird allgemein eine reelle Zahlgrösse ω verstanden, welche einer nicht identischen Gleichung von der Form genügt:

$$(1.) \quad a_0\,\omega^n + a_1\,\omega^{n-1} + \cdots + a_n = 0,$$

https://jamesrmeyer.com/pdfs/cantor-1874-ueber-eine-eigenschaft-des-inbegriffes.pdf

On a Property of the Set of all Real Algebraic Numbers

by Georg Cantor, 1874

By a real algebraic number one understands a real number ω which satisfies a non-constant equation of the form:

$$(1) \quad a_0\omega^n + a_1\omega^{n-1} + \ldots + a_n = 0$$

where n, a_0 , a_1 , ... a_n are integers: here the numbers n and a_0 are positive, and the coefficients a_0 , a_1 , ... a_n do not have any common factors,

according to size; in general, after all the numbers in (ω) up to a certain height $N = N_1$ have been enumerated and assigned to a definite position, the real algebraic numbers with the height $N = N_1 + 1$ follow them according to size; thus one obtains the set (ω) of all real algebraic numbers in the form:

$$\omega_1, \omega_2, \ldots \omega_\nu, \ldots$$

One can, with respect to this ordering, refer to the νth real algebraic number; not a single member of the set has been omitted.

§2.

Given any definition of an infinite sequence of mutually distinct real numbers,

$$(4) \quad \omega_1, \omega_2, \ldots \omega_\nu, \ldots$$

then in any given interval $(\alpha \ldots \beta)$ there is a number η (and consequently infinitely many such numbers) where it can be shown that it does not occur in the series (4); this shall now be proved.

https://jamesrmeyer.com/infinite/cantor-1874-uncountability-proof

Cantor-Dedekind Theorem: |real numbers| > |whole numbers |

- Cantor asked the question and drafted the proof
- Dedekind wrote the published proofs, including the less important one

- Dedekind did not fight for credit (except in a private note)
- Cantor defended the math against senior mathematicians

Thoughts

- Math is evolving (always)
- Math accomplishments come from collaboration

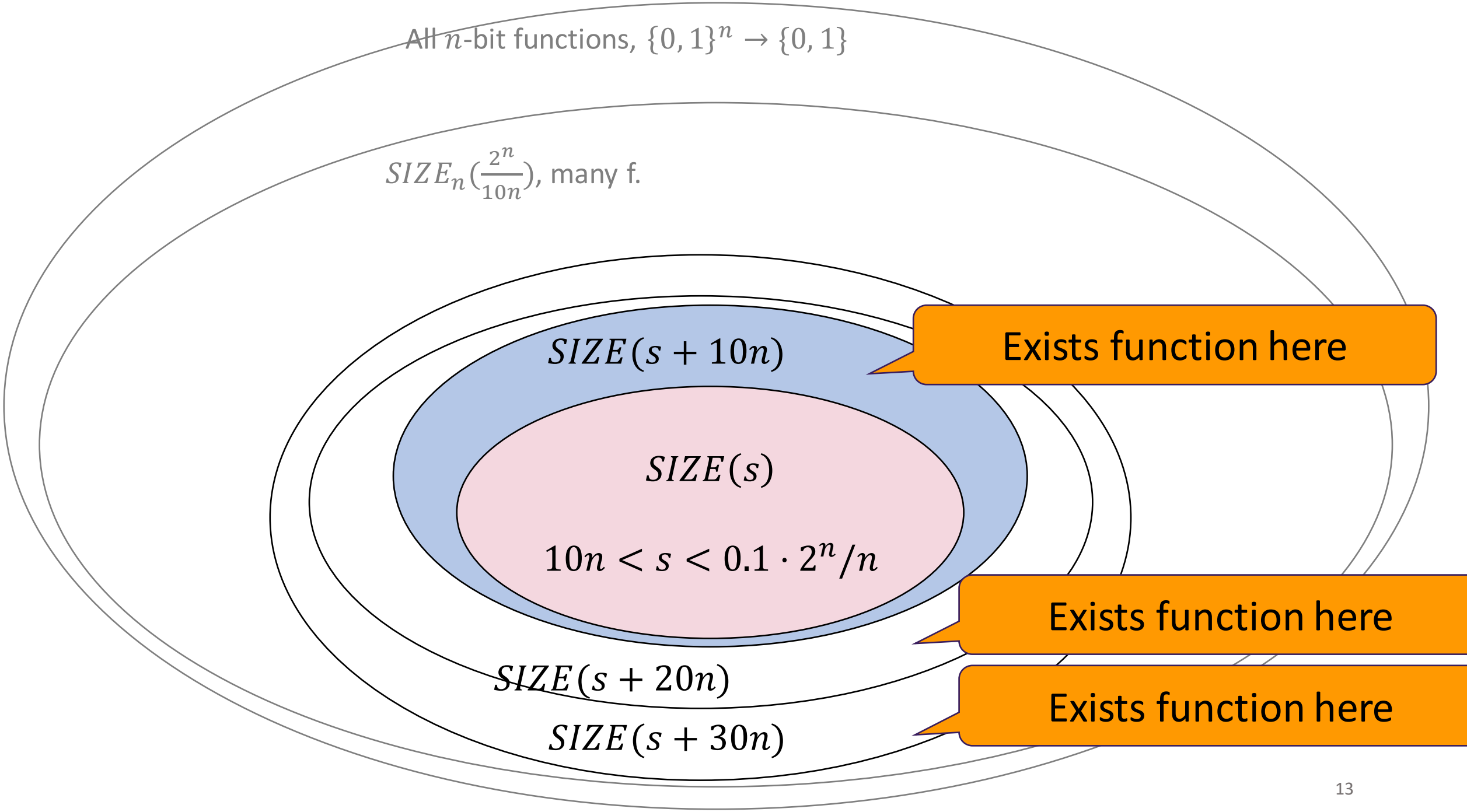
Wrapping up circuits

Recap: Size Hierarchy Theorem

Theorem 5.5 (Size Hierarchy Theorem)

For every sufficiently large n *and* $10n < s < 0.1 \cdot 2^n/n$,

$$SIZE_n(s) \subsetneq SIZE_n(s + 10n) \ .$$


All n-bit functions, $\{0,1\}^n \to \{0,1\}$
$SIZE_n(\frac{2^n}{10n})$, many f.
$SIZE(s+10n)$
$SIZE(s)$
$10n < s < 0.1 \cdot 2^n/n$
$SIZE(s+20n)$
$SIZE(s+30n)$
Exists function here
Exists function here
Exists function here

An Existential Proof

For any $s < \frac{2^n}{10n}$, there are many hard functions $f \notin SIZE(s)$.

Since most functions have large circuits, it is like: "finding hay in haystack".

But we don't know which!

"Complexity theory's Waterloo"
…
"We are unable to prove even a superlinear circuit lowerbound for any NP problem—the best we can do after years of effort is $\boldsymbol{4.5n - o(n)}$."

Physical Extended Church-Turing Thesis

"mathematically modeling the real world"

Physical Extended Church-Turing Thesis (PECTT):
If a function $F: \{0,1\}^n \to \{0,1\}^m$ *can be computed in the physical world using* s *amount of "physical resources" then it can be computed by a Boolean circuit program of roughly* s *gates.*

(time × space)
money
energy

Attempts to Refute PECTT

- DNA computers
- Continuous/real computers
- Relativity and time travel
- Humans
- Quantum computing

- (many)

Example: Cryptography

128-bit security:

We assume the problem cannot be solved by circuits with fewer than 2^{128} gates.

By PECTT, that takes 2^{128} amount of physical resource,

Eg, money, energy, physical time, space

Module 3: Turing Machines and Computability

Write regular expression to match (and only match) valid regular expressions

1169 votes

✓ 10 answers

270k views

Q Is there a regular expression to detect a valid regular expression?

Is it possible to detect a valid **regular expression** with another **regular expression**? If so please give example code below. ...

regex

psytek **9,231** asked Oct 5, 2008 at 17:07

https://stackoverflow.com/search?q=regular+expressions

Is there a regular expression to detect a valid

Ask Question

Products | Search... | Log in | Sign up

1169

Is it possible to detect a valid regular expression with another regular expression? If so please give example code below.

regex

Share Improve this question

Follow

edited Jan 8, 2016 at 12:58
xav
5,638 ● 7 ● 52 ● 58

asked Oct 5, 2008 at 17:07
psytek
9,231 ● 3 ● 19 ● 7

386 Who validates the validating regex? – bevacqua Jul 4, 2011 at 0:02

18 @Nico Community. – Janusz Lenar Jul 29, 2012 at 13:39

82 So your problem is validating a regex, you chose a regex for solving it. I wonder if the problem-number-increasing property of regexes is additive or multiplicative. It feels like 4 problems instead of 2 :) – abesto Nov 18, 2013 at 14:54

1077

```
/
^                                                # start of string
(                                                # first group start
  (?:
    (?:[^?+*{}()[\]\\|]+                         # literals and ^, $
     | \\.                                       # escaped characters
     | \[ (?: \^?\\. | \^[^\\] | [^\\^] )        # character classes
          (?: [^\]\\]+ | \\. )* \]
     | \( (?:\?[:=!]|\?<[=!]|\?>)? (?1)?? \)     # parenthesis, with recursive con
     | \(\? (?:R|[+-]?\d+) \)                    # recursive matching
     )
    (?: (?:[?+*]|\{\d+(?:,\d*)?\}) [?+]? )?      # quantifiers
  | \|                                           # alternative
  )*                                             # repeat content
)                                                # end first group
$                                                # end of string
/
```

This is a recursive regex, and is not supported by many regex engines. PCRE based ones should support it.

Write regular expression to match (and only match) valid regular expressions

Is it possible? Pumping lemma?

Let L = { e | e is a regular expression}. $\subseteq \{ '(', ')', "", \phi, 0, 1 \}^*$

$n_0 \in \mathbb{N}$, $w = "\underbrace{((((\cdots(}_{n_0} 1 \underbrace{))))\cdots)}_{n_0}" \in L$

$\forall\ xyz = w$

$\exists\ k = 0$ $\quad xy^0z = xz \notin L$

Write regular expression to match (and only match) valid regular expressions

Is it possible?

Let L = { e | e is a regular expression}.

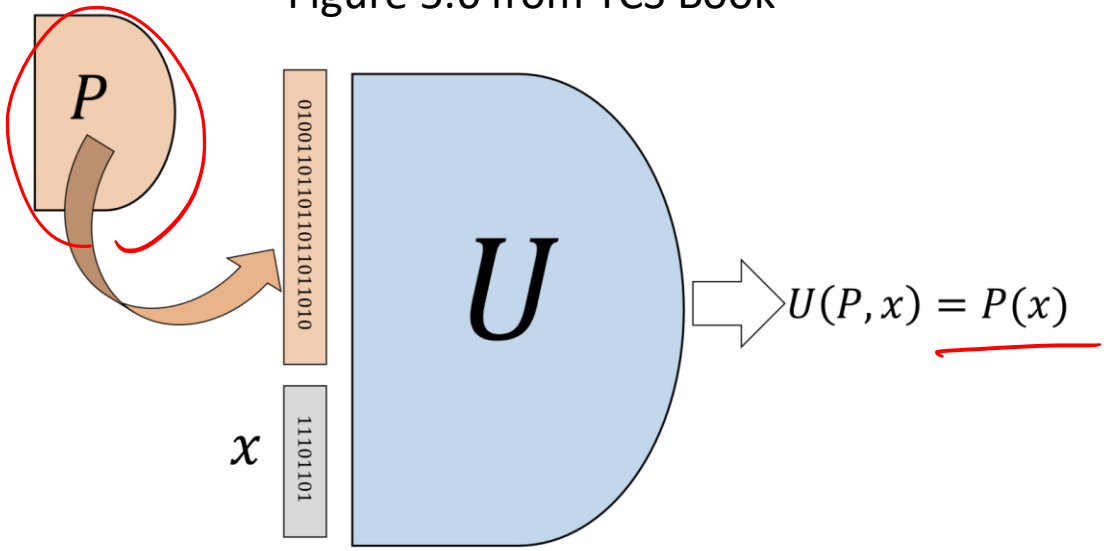
For any $n \in \mathbb{N}$, let $w = ({}^{n+1}0)^{n+1} \in L$.

For any $u + v \leq n, v \geq 1$, let $x = ({}^{u}, y = ({}^{v}$.

We have $xyz = w$, but $xy^k z \notin L$ for $k = 0$.

Recall, Class 12: Universal Circuit/Program

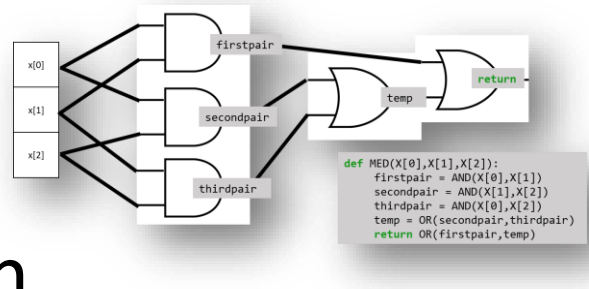
Figure 5.6 from TCS Book



program U takes a program description P and input x as its input, and "simulates" running P on x:

$$U(P,x) = P(x)$$

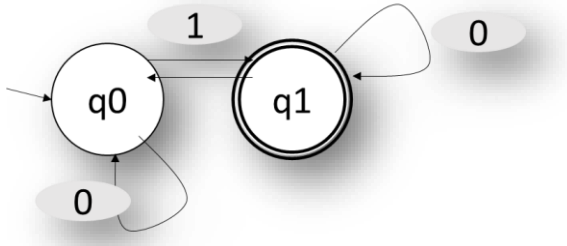
Computation models (so far)



Boolean circuits:

☺ Can compute every function on fixed input length

☹ Need a different circuit for each input length.



DFA (and RE, NFA):

☺ Can describe one algorithm/method for all inputs

☹ It is limited in how it accesses its inputs (one pass)

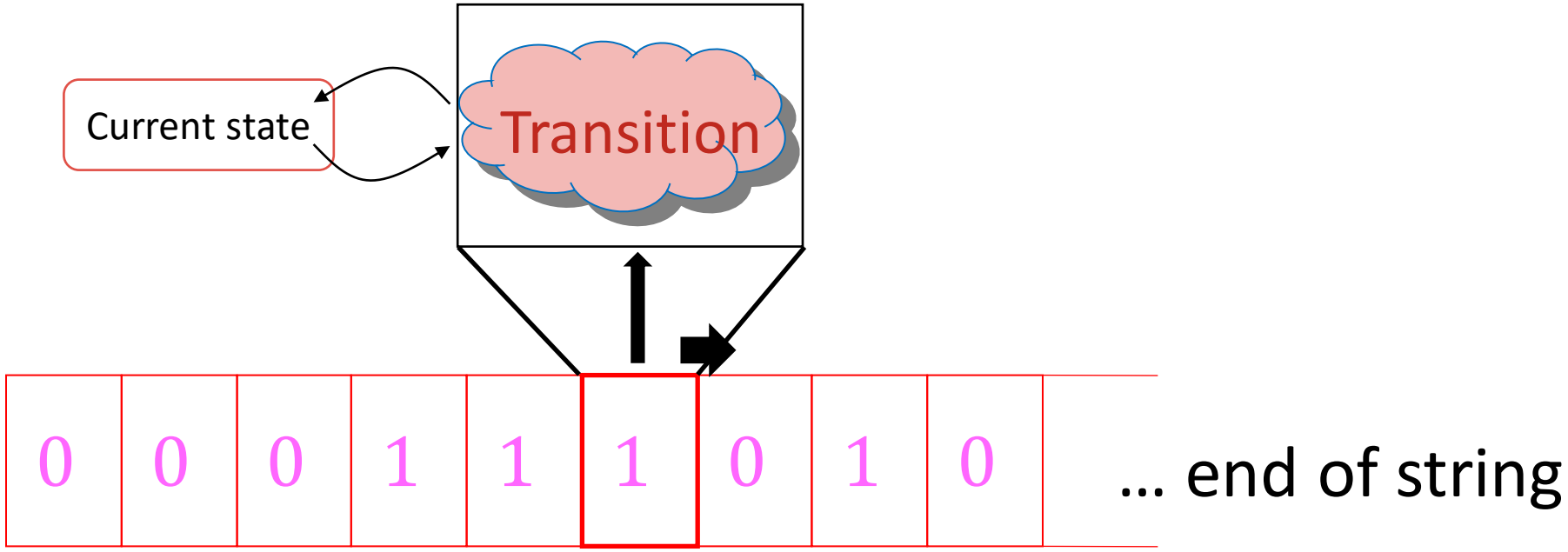
☹ It is limited in how much memory it has (constant)

Can we get the best of both

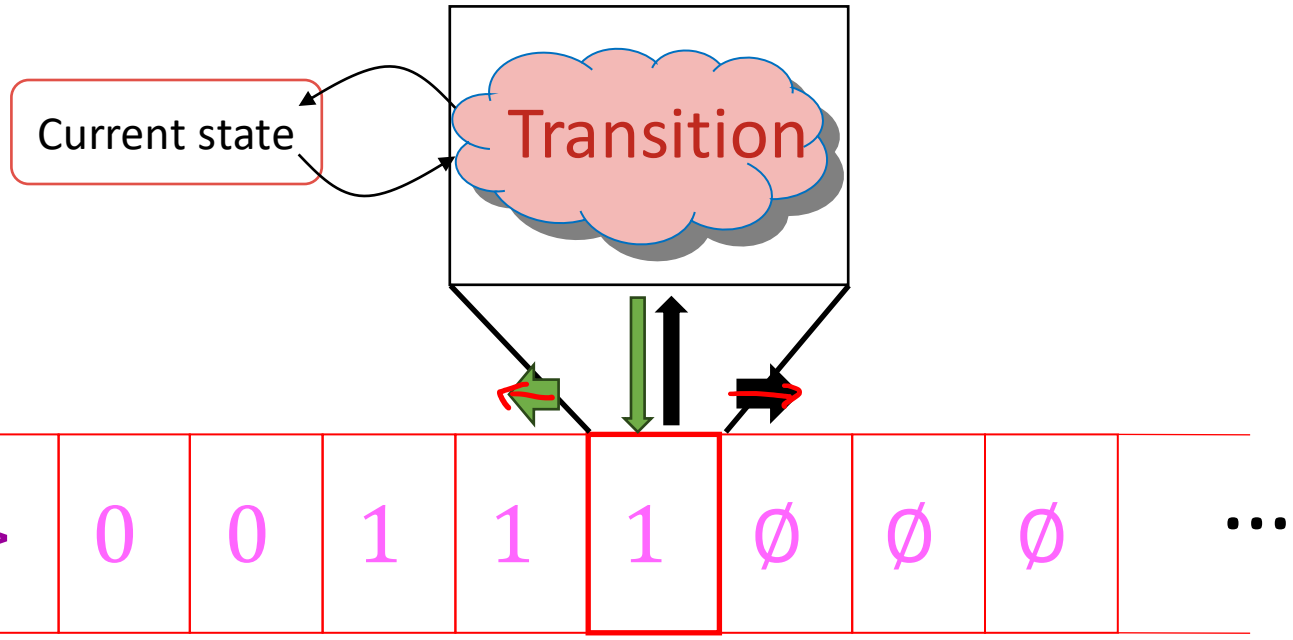
Turing Machines!

☺ A single algorithm for all input lengths
☺ Can read its input back and forth
☺ Can store data and read back (storage as memory)

How DFA worked



Turing Machine



Tape initially contains the input followed by blanks

If TM "halts", → tape contains the output

Semi-infinite tape (memory)

Turing Machines

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The "computable" numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbrous technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development

https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

A. M. Turing (1950) Computing Machinery and Intelligence. *Mind 49:* 433-460.

COMPUTING MACHINERY AND INTELLIGENCE

By A. M. Turing

1. The Imitation Game

I propose to consider the question, "Can machines think?" This should begin with definitions of the meaning of the terms "machine" and "think." The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous, If the meaning of the words "machine" and "think" are to be found by examining how they are commonly used it is difficult to escape the conclusion that the meaning and the answer to the question, "Can machines think?" is to be sought in a statistical survey such as a Gallup poll. But this is absurd. Instead of attempting such a definition I shall replace the question by another, which is closely related to it and is expressed in relatively unambiguous words.

https://courses.cs.umbc.edu/471/papers/turing.pdf

4. Digital Computers

The idea behind digital computers may be explained by saying that these machines are intended to carry out any operations which could be done by a human computer. The human computer is supposed to be following fixed rules; he has no authority to deviate from them in any detail. We may suppose that these rules are supplied in a book, which is altered whenever he is put on to a new job. He has also an unlimited supply of paper on which he does his calculations. He may also do his multiplications and additions on a "desk machine," but this is not important.

If we use the above explanation as a definition we shall be in danger of circularity of argument. We avoid this by giving an outline. of the means by which the desired effect is achieved. A digital computer can usually be regarded as consisting of three parts:

(i) Store.

(ii) Executive unit.

(iii) Control.

Defining Turing Machines

Follow simple rules

Remember what you are doing

Turing Machine Model

A *Turing Machine,* is defined by (Σ, k, δ):

$\boldsymbol{k} \in \mathbb{N}$: a finite number of states

Σ : alphabet – finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\{0, 1, \triangleright, \phi, a, b, c..\}$

$\boldsymbol{\delta}$: transition function

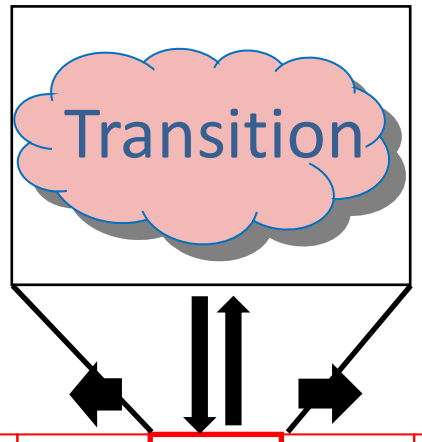
$$\boldsymbol{\delta}: \underbrace{[k]}_{\text{state}} \times \underbrace{\Sigma}_{\text{read symb}} \to \underbrace{[k]}_{\text{state}} \times \underbrace{\Sigma}_{\text{write symb}} \times \underbrace{\{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}}_{\text{tape move}}$$

Left, **R**ight, **S**tay, **H**alt

A **(deterministic)** ***finite automaton*** over alphabet {0,1} is a tuple (C, T, S) where:

1. C --- the number of *states*
2. $T: [C] \times \{0,1\} \to [C]$ --- a transition function
3. $S \subseteq [C]$ --- the set of accept states

Turing Machine



Tape initially contains the input followed by blanks

▷	0	0	1	1	1	∅	∅	∅	…

location 0

Semi-infinite tape (memory)

blank or empty

A *Turing Machine,* is defined by (Σ, k, δ):

$\boldsymbol{k} \in \mathbb{N}$: a finite number of states

Σ : alphabet $-$ finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [k] \times \Sigma \to [k] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$$

Execution of a TM

Definition. The execution of a TM, $M = (\Sigma, k, \delta)$ on input $x \in \{0,1\}^*$ is this process:

1. Initialize T as $\triangleright, x_0, x_1, \ldots, x_{n-1}, \emptyset, \emptyset, \emptyset, \ldots$ where $n = |x|$.

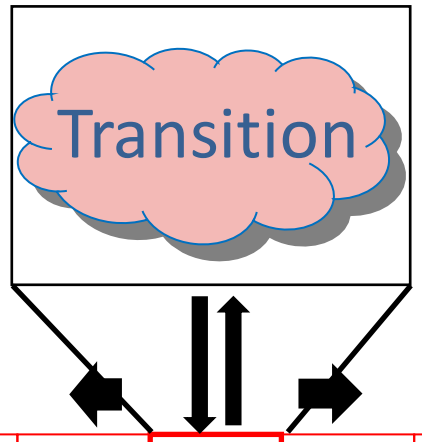
   That is, $T[0] = \triangleright$, $T[i+1] = x_i$ for $i \in [n]$, and $T[i] = \emptyset$ for $i > n$.
2. Initialize two natural number variables, $i = 0$, $s = 0$.

   locat tape state
3. **repeat**
   1. $(s', \sigma', D) = \boldsymbol{\delta}(s, T[i])$
   2. $s := s', T[i] := \sigma'$
   3. if $D = \mathbf{R}$: $i := i + 1$

      if $D = \mathbf{L}$: $i := \max\{i - 1, 0\}$

      if $D = \mathbf{H}$: **break**

Turing Machine



Tape initially contains the input followed by blanks

▷	0	0	1	1	1	∅	∅	∅	…

Semi-infinite tape (memory)

If transition "halts", → tape contains the output

Output of executing TM

Definition. The **output** of the execution of a TM, $M = (\Sigma, k, \delta)$ on input $x \in \{0,1\}^*$ is the result of this process:

1. Initialize T as $\triangleright, x_0, x_1, \dots, x_{n-1}, \emptyset, \emptyset, \emptyset, \dots$ where $n = |x|$.
   That is, $T[0] = \triangleright$, $T[i+1] = x_i$ for $i \in [n]$, and $T[i] = \emptyset$ for $i > n$.
2. Initialize two natural number variables, $i = 0, s = 0$.
3. **repeat**
   1. $(s', \sigma', D) = \boldsymbol{\delta}(s, T[i])$
   2. $s := s', T[i] := \sigma'$
   3. if D = $\mathbf{R}$: $i := i + 1$
      if D = $\mathbf{L}$: $i := \max\{i - 1, 0\}$
      if D = $\mathbf{H}$: **break**
4. If the process finishes (the repeat breaks), the result of this process is:
   $M(x) = T[1], \dots, T[m]$ where $m > 0$ is the smallest integer, $T[m+1] \notin \{0,1\}$.

Otherwise (i.e., the machine does NOT halt), then define $M(x) = \perp$.

[warning: $\perp$ is not an actual output]

Simulating Turing Machines

http://morphett.info/turing/turing.html

```
0 0 0 r 0
0 1 1 r b
0 _ _ * halt-accept
b 0 0 r b
b 1 1 r b
b _ _ * halt-reject
```

Plan

Turing machine

Computability

Textbook [TCS] Section 5 and 7

https://introtcs.org/public/lec_06_loops.html
Loops and infinity

HW 5 due this Friday

Quiz 7 coming soon